$\pm 3.4 \times 10^{38}$ and $\pm 1.2 \times 10^{-38}$, respectively. The freed bit patterns allow three special classes of numbers: (1) ± 0 is defined as all of the mantissa and exponent bits being zero. (2) $\pm \infty$ is defined as all of the mantissa bits being zero, and all of the exponent bits being one. (3) A group of very small *unnormalized* numbers between $\pm 1.2 \times 10^{-38}$ and $\pm 1.4 \times 10^{-45}$. These are lower precision numbers obtained by removing the requirement that the leading digit in the mantissa be a one. Besides these three special classes, there are bit patterns that are not assigned a meaning, commonly referred to as NANs (Not A Number).

The IEEE standard for double precision simply adds more bits to both the mantissa and exponent. Of the 64 bits used to store a double precision number, bits 0 through 51 are the mantissa, bits 52 through 62 are the exponent, and bit 63 is the sign bit. As before, the mantissa is between one and just under two, i.e., $M = 1 + m_{51}2^{-1} + m_{50}2^{-2} + m_{49}2^{-3} \cdots$. The 11 exponent bits form a number between 0 and 2047, with an offset of 1023, allowing exponents from 2^{-1023} to 2^{1024}. The largest and smallest numbers allowed are $\pm 1.8 \times 10^{308}$ and $\pm 2.2 \times 10^{-308}$, respectively. These are incredibly large and small numbers! It is quite uncommon to find an application where single precision is not adequate. You will probably never find a case where double precision limits what you want to accomplish.

Number Precision

The errors associated with number representation are very similar to quantization errors during ADC. You *want* to store a continuous range of values; however, you *can* represent only a finite number of quantized levels. Every time a new number is generated, after a math calculation for example, it must be rounded to the nearest value that can be stored in the format you are using.

As an example, imagine that you allocate 32 bits to store a number. Since there are exactly $2^{32} = 4{,}294{,}967{,}296$ different bit patterns possible, you can represent exactly 4,294,967,296 different numbers. Some programming languages allow a variable called a **long integer**, stored as 32 bits, fixed point, two's complement. This means that the 4,294,967,296 possible bit patterns represent the integers between -2,147,483,648 and 2,147,483,647. In comparison, single precision floating point spreads these 4,294,967,296 bit patterns over the much larger range: -3.4×10^{38} to 3.4×10^{38}.

With fixed point variables, the gaps between adjacent numbers are always *exactly one*. In floating point notation, the gaps between adjacent numbers vary over the represented number range. If we randomly pick a floating point number, the gap next to that number is approximately *ten million times smaller* than the number itself (to be exact, 2^{-24} to 2^{-23} times the number). This is a key concept of floating point notation: large numbers have large gaps between them, while small numbers have small gaps. Figure 4-3 illustrates this by showing consecutive floating point numbers, and the gaps that separate them.